Lexing, Parsing & Semantic Analysis

CG

November 7, 2026

Contents

1 Lexical analysis **2**
- 1.1 The lexer in a compiler . . . 2
- 1.2 Lexemes, patterns and tokens . . . 3
- 1.3 Symbol tables . . . 4
- 1.4 Recognizing tokens . . . 4
- 1.5 A lexer in Rascal . . . 7
- 1.6 Token implementation and screening . . . 8
- 1.7 Exercises . . . 9

2 Parsing **11**
- 2.1 Errors . . . 11
- 2.2 Top-down parsing . . . 12
- 2.3 FIRST and FOLLOW . . . 14
- 2.4 LL(1) grammars and the parsing table . . . 16
- 2.5 Bottom-up parsing . . . 19
- 2.6 LR parsing . . . 20
- 2.7 Precedence climbing . . . 23
- 2.8 Exercises . . . 24

3 Abstract syntax trees and attribute grammars **25**
- 3.1 Abstract syntax trees . . . 25
- 3.2 Attribute grammars . . . 25
- 3.3 Building ASTs . . . 27
- 3.4 Exercises . . . 30

4 Semantic analysis **30**
- 4.1 Static analysis . . . 31
- 4.2 Identification of identifiers . . . 32

4.3 Dynamic analysis 35
4.4 More static analyses 35
4.5 Exercises 36

1 Lexical analysis

A compiler reads a program as a sequence of characters. The first phase, the lexical analyzer or lexer, groups the characters into tokens and passes them to the parser. This section follows Aho et al. [1, Ch. 3], Wilhelm, Seidl and Hack [5, Ch. 2–3] and Nystrom [3, Ch. 4–5].

1.1 The lexer in a compiler

Figure 1 shows where the lexer sits. The parser asks for one token at a time (`getNextToken`); the lexer reads as many characters as it needs and returns the next token. Both phases use the symbol table, which keeps information about the names of the program. The later phases (semantic analysis, optimization and code generation) work on what the parser produces.

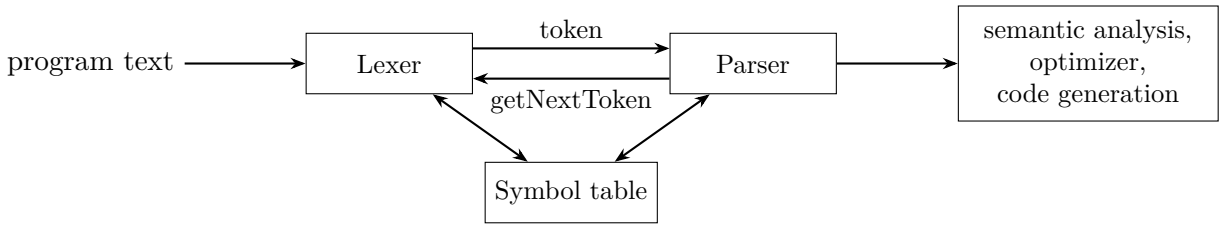


Figure 1: The lexer and the parser in a compiler.

The lecture starts from this C program:

```
int a = 10;
int b = 20;
int c = 20;
float f = 928.2332;
char b = 'b';
if(a == b){
  printf("Hello, World! \"");
}else if(b!=c){
  printf("Hello, World! \n Hello, World!");
}else{
  printf("Hello\n");
}
```

The lexer does two things with it. The **scanner** part removes the characters that do not matter for the structure: spaces, line breaks used as separators, and comments such as `// first condition` or `/* ... */`. The second part recognizes tokens: `int` is a type, `a` is an identifier, `10` is a number, `"Hello, World! \""` is a string (the escaped quote does not end it). The first line becomes `int Identifier = Number ;`.

The lexer also detects lexical errors: characters that do not start any token. In the lecture's example `char b = 'b';` is fine, but the next line mixes letters and digits with symbols that are

not in the alphabet of C. No pattern starts with those symbols, so the lexer reports an error there. A lexer only sees characters and local patterns; it does not know whether `b` was declared twice (it was) or whether `a == b` compares values of compatible types. Those are problems for the parser and for semantic analysis.

1.2 Lexemes, patterns and tokens

Definition 1.1. A **lexeme** is a sequence of characters that matches a pattern useful for the language. A **pattern** is a specification of the form that lexemes take. A **token** is a pair $\langle token_name, value \rangle$, where the token name represents a pattern.

In the WFF language of the previous block, the rule `Operator ::=` $\vee$ `|` $\wedge$ `|` $\Rightarrow$ describes the pattern of the token `operator`. The formula $(p \vee q) \Rightarrow (p \wedge q) \Rightarrow (\neg q \wedge \neg p)$ becomes

$$(p \ \textit{operator}\ q)\ \textit{operator}\ (p\ \textit{operator}\ q)\ \textit{operator}\ (\neg q\ \textit{operator}\ \neg p).$$

Here the token name alone loses information: to execute the program later, the compiler needs to know which operator each one was. The value of the token keeps it, and the lexer also records where the token appears. Table 1 lists the operators of the formula, written without spaces, with their position and their type.

Lexeme	Row	Column	Type
$\vee$	1	3	BinOp
$\Rightarrow$	1	6	BinOp
$\wedge$	1	9	BinOp
$\Rightarrow$	1	12	BinOp
$\wedge$	1	16	BinOp

Table 1: Operator tokens of $(p \vee q) \Rightarrow (p \wedge q) \Rightarrow (\neg q \wedge \neg p)$.

Most languages use the same token classes (Table 2).

Class	Content
Identifier	sequences of letters and digits not otherwise recognized
Number	numeric constants, usually refined by their type
String	string literals, delimited by `"..."`
Operators	comparison, arithmetic and boolean operators, usually refined by their type
Control	control structures such as `if` and loops, refined by their kind
Delimiters	parentheses and punctuation, usually each its own token

Table 2: Token classes.

In Rascal a token can be a constructor with its name, its lexeme and its position:

```
data Token = token(str name, str lexeme, int line, int col);
```

1.3 Symbol tables

Definition 1.2. A **symbol table** is a data structure used by the compiler to associate the symbols of the program with information about their declaration: the name, the type, the scope, the memory address, the location in the source, the value and any additional information.

For `Area = pi * r ** 2` the lexer produces `<Identifier, 1> <Equal> <Identifier, 2> <Star> <Identifier, 3> <DoubleStar> <Number, 2>`, where the value of each identifier token is its entry in the symbol table:

Entry	Name	Value	Location
1	`Area`		line 1, column 1
2	`pi`	3.1416	line 1, column 8
3	`r`	reference	line 1, column 13

`**` is a single token, not two `*`: the lexer takes the longest lexeme that matches a pattern.

Symbol tables are usually implemented as a linear structure (a list, an array or a stack) of hash tables, one per scope: `symbolTables = [st1, st2, ..., stn]`. Figure 2 shows the tables for the lecture's example:

```
extern double bar(double x);
double foo(int count) {
  double sum = 0.0;
  for (int i = 1; i <= count; i++)
    sum += bar((double) i);
  return sum;
}
```

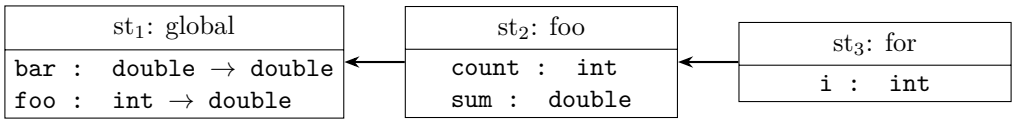


Figure 2: One table per scope. A name is looked up from the innermost table outward.

The parameter `x` of `bar` only appears in a declaration, so it needs no entry of its own in this example. Section 4 uses the same structure to resolve names during semantic analysis.

1.4 Recognizing tokens

Lookahead. The lexer cannot always decide where a token ends when it reads the last character of the token; sometimes it needs the next one. The usual algorithm keeps the input in a buffer with two pointers (Figure 3). *beginning* marks the start of the current lexeme and *forward* advances until a pattern is matched. Then the token is generated, *beginning* moves to *forward*, and the process continues with the next lexeme.

From patterns to one automaton. Token patterns form regular languages, so DFAs recognize them. The construction combines the automata of all patterns:

1. build a DFA $\mathcal{A}_i = \langle Q_i, \Sigma, q_{0_i}, F_i, \delta_i \rangle$ for each regular expression r_i that specifies a pattern;

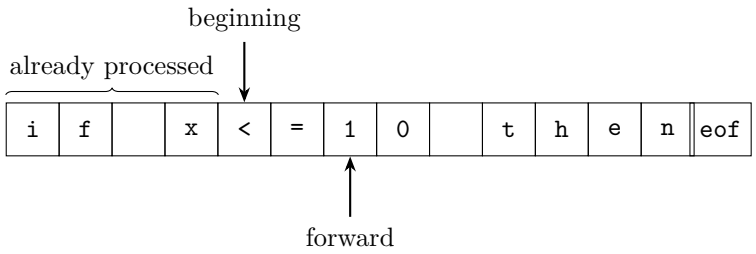


Figure 3: The two pointers while reading `<=`. *forward* is one position past the end of the lexeme: the lexer read `1` to know that `<=` was finished.

2. join them with a new initial state q_0 and λ-transitions to the initial state of each $\mathcal{A}_i$: $Q = \{q_0\} \cup Q_1 \cup \cdots \cup Q_n$, $F = F_1 \cup \cdots \cup F_n$, $\Delta = \{(q_0, \lambda, q_{0_i}) \mid 1 \leq i \leq n\} \cup \delta_1 \cup \cdots \cup \delta_n$;
3. apply the set construction to get a DFA. A word belongs to the class i when it leads to a final state of $\mathcal{A}_i$.

This is the union construction of the previous block. When a word reaches final states of two classes at once, a priority decides, and the lexer always keeps reading while the word can still grow (longest match).

Example. The lecture's grammar

```
Stmt ::= if Expr then Stmt | if Expr then Stmt else Stmt
Expr ::= Term Op Term | Term
Term ::= id | number
Op   ::= = | <> | <= | >= | < | >
```

has the tokens `if`, `then`, `else`, `id`, `number` and `Op`. The program

```
if iffyNum <> elsewhereNum then
   iffyNum
else
   elsewhereNum
```

becomes `IF ID OP_NE ID THEN ID ELSE ID`. Two details matter. The identifier `iffyNum` starts with `if` and `elsewhereNum` with `else`, and the lexer must not split them. And `<>` is one token, not `<` followed by `>`.

Identifiers and reserved words. The pattern of identifiers also matches `if`. The lecture gives two ways to separate them. The first recognizes reserved words with their own states: after `i` and `f`, if the next character is not a letter or digit, return `<IF>`; otherwise continue as an identifier (Figure 4). The second installs the reserved words in the symbol table before scanning, as `<IF, if_value>`. The lexer reads the longest identifier-shaped lexeme and looks it up: a reserved word becomes its own token, any other name is installed as an identifier (`return(installId())`). The second way keeps the automaton small, and it is what the Rascal lexer below does.

Relational operators. Figure 5 is the lecture's automaton for `<`, `<=`, `<>`, `=`, `>` and `>=`. After reading `<` the automaton only knows that the token is `<` when it reads the following character and sees that it is not `=` or `>`. The states marked with $*$ return that extra character to the input.

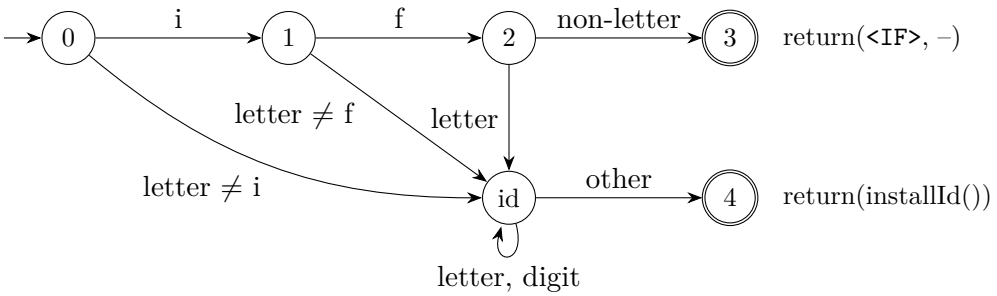


Figure 4: Recognizing `if` and identifiers with separate states. In states 3 and 4 the last character read is not part of the token and goes back to the input.

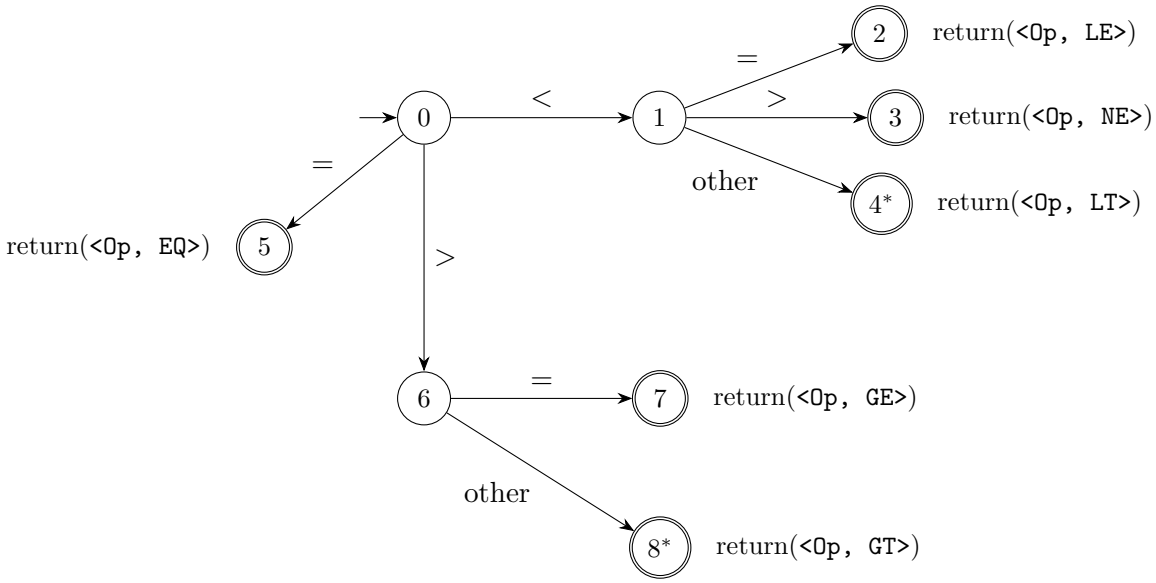


Figure 5: The automaton for relational operators. $*$ marks the states that read one character too many.

The automaton becomes code by giving each state a case. The lecture writes it as a `switch` on the state inside a loop; the version below collapses the states into the checks they make:

```
tuple[str, int] relop(str s) {
  str next = size(s) > 1 ? s[1] : "";
  switch (s[0]) {
    case "\<": {
      if (next == "=") return <"LE", 2>;
      if (next == "\>") return <"NE", 2>;
      return <"LT", 1>;
    }
    case "=": return <"EQ", 1>;
    case "\>": {
      if (next == "=") return <"GE", 2>;
      return <"GT", 1>;
    }
  }
  return <"FAIL", 0>;
}
```

1.5 A lexer in Rascal

Putting the pieces together gives a hand-written lexer in the style of Nystrom's `scanTokens`. Each iteration of the loop matches one lexeme at the beginning of the remaining input, from the most specific pattern to the least specific, and advances the position. Separators and comments are matched and dropped; reserved words are looked up in a set; any other character is a lexical error.

```
set[str] reserved = {"if", "then", "else"};

list[Token] scan(str src) {
  list[Token] tokens = [];
  int line = 1;
  int col = 1;
  while (src != "") {
    str lexeme = "";
    str name = "";
    if (/^<ws:[ \t]+>/ := src) {
      lexeme = ws;
    } else if (/^\n/ := src) {
      src = src[1..]; line += 1; col = 1; continue;
    } else if (/^<c:\/\/[^\n]*>/ := src) {
      lexeme = c;
    } else if (/^<w:[a-zA-Z][a-zA-Z0-9]*>/ := src) {
      lexeme = w;
      name = w in reserved ? toUpperCase(w) : "ID";
    } else if (/^<n:[0-9]+(\.[0-9]+)?>/ := src) {
      lexeme = n; name = "NUM";
    } else if (/^[\<\>=]/ := src) {
      <name, len> = relop(src);
      lexeme = src[..len];
      name = "OP_" + name;
    } else if (/^<p:\*\*|[*+\-\/;,(){}]>/ := src) {
      lexeme = p; name = "PUNCT";
    } else {
      lexeme = src[0]; name = "ERROR";
    }
    if (name != "") tokens += [token(name, lexeme, line, col)];
    src = src[size(lexeme)..];
    col += size(lexeme);
  }
  return tokens;
}
```

Each regular expression in the `if` chain is one pattern. The pattern for identifiers takes the longest run of letters and digits, so `iffyNum` is one lexeme, and only then is it compared with the reserved words. The tests in `Lexer.rsc` check the `iffyNum` program, the relational operators, the positions in `Area = pi * r ** 2` and the WFF operators of Table 1.

Generated lexers. This paragraph goes beyond the slides. Tools such as `lex` and `flex` take the list of regular expressions with an action for each and generate the combined DFA automatically, with the same rules: longest match, and the first pattern wins when two match the same lexeme. In Rascal, `lexical` definitions play this role, as in the grammars of the previous block. A hand-written

lexer gives more control over error messages and over irregular rules, such as indentation in Python.

1.6 Token implementation and screening

Lexers usually give each token name a number, so that tokens are compared as integers. For the `if` grammar the lecture uses

ELSE	EQ	G	GE	ID	IF	L	LE	NE	NUM	THEN
260	261	262	263	264	265	266	267	269	270	273

Numbers above 255 leave room for single characters, which can be their own token with their character code.

Wilhelm, Seidl and Hack split the lexer into two steps. **Scanning** cuts the input into lexemes and classifies all of them, separators included. **Screening** drops the separators and comments, recognizes reserved words and replaces each identifier with its index in the symbol table. For

```
int a, b;
a = 42;
b = a * a - 7;
```

scanning gives

```
Int("int") Sep(" ") Id("a") Com(",") Sep(" ") Id("b") Sem(";") Sep("\n") ...
```

and screening gives

```
Int() Id(1) Com() Id(2) Sem()
Id(1) Bec() Intconst(42) Sem()
Id(2) Bec() Id(1) Mop(Mul) Id(1) Aop(Sub) Intconst(7) Sem()
```

(The slide writes `Aop("")` for the minus sign in the scanning line; it is `Aop("-")`.) A screening step on top of the Rascal lexer:

```
tuple[list[str], map[str, int]] screen(list[Token] tokens) {
  map[str, int] table = ();
  list[str] out = [];
  for (t <- tokens) {
    if (t.name == "ID") {
      if (t.lexeme notin table) table[t.lexeme] = size(table) + 1;
      out += ["Id(<table[t.lexeme]>)"];
    } else if (t.name == "NUM") {
      out += ["Intconst(<t.lexeme>)"];
    } else {
      out += [t.lexeme];
    }
  }
  return <out, table>;
}
```

Responsibilities of the lexer. To summarize the lecture's list: the lexer generates tokens; it detects errors in the characters of the program, not in its structure or meaning; it recognizes

patterns while scanning the text (separators, comments, pragmas, reserved symbols, constants, strings, identifiers); and it classifies symbols, places them in the right table and associates them with their value.

1.7 Exercises

Exercise 1.3. Generate the tokens of this JavaScript program:

```
let boolean = true;
let antiboolean = !true;
if(boolean !== antiboolean) {
  true;
  if(antiboolean != boolean) {
    false;
  }
}
```

Solution.

```
Let Identifier Equal Bool ;
Let Identifier Equal Not Bool ;
If ( Identifier SInequality Identifier ) {
  Bool ;
  If ( Identifier LInequality Identifier ) {
    Bool ;
  }
}
```

`boolean` is an identifier, not a type: JavaScript has no reserved word `boolean`. `!==` (strict inequality) and `!=` (loose inequality) are different tokens, and by longest match the lexer reads `!==` whole instead of `!=` followed by `=`.

Exercise 1.4. Generate the tokens of this Python program:

```
price_list = dict(milk=23.25, bread=340, eggs=1)
shopping_cart = {'eggs':5, 'bread':170}
for item in shopping_cart:
  if(price_list.get(item)):
    cost = price_list[item]
    quantity = shopping_cart[item] / price_list[item]
    print(quantity)
  else:
    print(f'Item {item} from another store')
```

Solution. The lecture's answer:

```
Identifier Equal Identifier(Identifier Equal Float, Identifier Equal Integer,
    Identifier Equal Integer)
Identifier Equal {String : Integer, String : Integer}
For Identifier In Identifier :
  If(Identifier.Identifier(Identifier)) :
    Identifier Equal Identifier[Identifier]
    Identifier Equal Identifier[Identifier] Slash Identifier[Identifier]
```

```
    Identifier(Identifier)
  Else :
    Identifier(String Identifier String)
```

`dict` and `print` are identifiers: in Python they are built-in functions, not reserved words. The indentation is also significant in Python, so a real Python lexer emits `INDENT` and `DEDENT` tokens where the level changes. The f-string can be one token or, as in the answer, split into its literal parts and the identifier inside the braces.

Exercise 1.5. Generate the tokens of the two Fortran statements `DO 5 I = 1.25` and `DO 5 I = 1,25`.

Solution. Fortran ignores blanks, so the first statement is an assignment to the variable `DO5I`, with the tokens `Identifier Equal Float`. The second is a loop that ends at label 5 with `I` from 1 to 25. Its tokens are `Do Int Identifier Equal Int , Int`. The two statements are the same up to the comma, so the lexer cannot classify `DO` until it has looked ahead past the `=`:

```
list[str] fortran(str stmt) {
  str s = replaceAll(stmt, " ", "");
  if (/^DO<label:[0-9]+><var:[A-Z][A-Z0-9]*>=<from:[0-9]+>,<to:[0-9]+>$/ := s) {
    return ["Do", "Int", "Identifier", "Equal", "Int", "Comma", "Int"];
  }
  if (/^<id:[A-Z][A-Z0-9]*>=[0-9]+\.[0-9]+$/ := s) {
    return ["Identifier", "Equal", "Float"];
  }
  return ["Error"];
}
```

Exercise 1.6. Tokenize the following inputs and draw the automaton that recognizes the reserved words in each one: (a) the C declaration `struct myStructure { int myNum; char myLetter; };`; (b) the XML document

```
<?xml version="1.0" encoding="UTF-8"?>
<note>
<to>Tove</to>
<from>Jani</from>
<heading>Reminder</heading>
<body>Don't forget me this weekend!</body>
</note>
```

(c) the SQL query `SELECT CustomerName, City FROM Customers;`; (d) the Racket expression `(match #(1 2) [(list 1 2) 'list] [(vector 1 2) 'vector])`, whose value is `'vector`.

Solution. (a) `Struct Identifier { Int Identifier ; Char Identifier ; } ;`. The automaton for `struct` is a chain $0 \xrightarrow{s} 1 \xrightarrow{t} 3 \xrightarrow{r} 4 \xrightarrow{u} 5 \xrightarrow{c} 6 \xrightarrow{t} 7$, and state 7 returns `<STRUCT>` only if the next character cannot continue an identifier (here it is a space before `myStructure`). (b) The lexer recognizes the declaration `<?xml ...?>`, start tags (`<` name `>`), end tags (`</` name `>`), attributes with their string values, and text between tags: `Decl StartTag(note) StartTag(to) Text(Tove) EndTag(to) ...`. The text `Don't forget me this weekend!` is one token even though it contains spaces, because inside an element the lexer is in a different mode. (c) `Select Identifier , Identifier From Identifier ;`. (d)

```
( Identifier VectorStart Number Number )
  [ ( Identifier Number Number ) Quote Identifier ]
  [ ( Identifier Number Number ) Quote Identifier ] )
```

`#(` is one token that opens a vector, and `'` is the quote. `match`, `list` and `vector` are identifiers.

Exercise 1.7. The grammar `E ::= (a..z)+ | (0..9)+ | (sum L) | (times L)`, `L ::= E | L E` describes S-expressions with two operators. Which tokens does it need, and which prefixes must the lexer recognize to tell the reserved words apart from identifiers?

Solution. The tokens are numbers, identifiers, the two operators and the parentheses. The words `sum` and `times` are also identifiers, so the lexer needs states for all their prefixes: s, su, sum, t, ti, tim, time and times. On a character that cannot continue the word (a space or a parenthesis), a state that holds a complete reserved word returns `<Op, +>` or `<Op, *>`; any other state returns an identifier. With the operators recognized, the grammar can be rewritten over tokens as

```
E ::= v | n | (+ L) | (* L)
```

Exercise 1.8. Lex `y != 4 // stop` by hand with the lexer of this section. What does it do with `!`?

Solution. `ID(y)`, then `!` matches no pattern and becomes `ERROR(!)`, then `=` is `OP_EQ`, `4` is `NUM`, and the comment is dropped. To support `!=` the relational automaton needs a state after `!` that returns `<Op, NE>` on `=`.

2 Parsing

Parsers analyze the syntax of programs: they make sure that the input follows the syntactic rules (the grammar) of the language and build its structure. Their role is also to detect errors. This section follows Aho et al. [1, Ch. 4], Lee [2, Sec. 2.8–2.11] and Nystrom [3, Ch. 6–7].

2.1 Errors

The lecture uses this function, written in a made-up language, to show four kinds of errors:

```
function testFunction(Int a, Int b) -> Int {
  if (b = a) {
    print("equal numbers a")
  } elfi (a > b)
    a = a * (b + c * d
  elif (a != b) {
    b = (a + b) / (b - a * c);
  } else
    "a is not is not a real number"
}
```

Kind	Meaning	In the example
Lexical	misspellings	`elfi` instead of `elif`
Syntactic	misplaced elements in the program text	the parenthesis opened in `(b + c * d` is never closed
Semantic	type mismatches, for example in function returns	the function returns `Int` but the last branch is a string
Logical	incorrect reasoning by the programmer	`b = a` instead of `b == a`

The parser is responsible for the syntactic errors. It has four independent responsibilities, of which only the first is mandatory: locating and reporting the error, diagnosing it, correcting it, and continuing the analysis after it.

Panic-mode recovery. The parser discards tokens until it finds a synchronizing token, such as `;` or `}`, and continues from there. It skips the discarded tokens without checking them for errors. In the example, after the unclosed parenthesis the parser can skip to the next `elif`. A parser that only reports and stops is the extreme case: `panicked at 'unmatched parenthesis', testFunction.lang:5`.

Phrase-level recovery. The parser fixes the error where it is detected with a local correction, replacing the processed prefix by a valid one. In

```
if (a == b)
  a = a * ((b + c) * d)
else if (a != b) {
```

the parser detects a missing `;` at the end of the second line and inserts it. This works when the error is at the point where it is detected. It struggles when the real error happened earlier: a missing `(` is only detected at the unmatched `)`. The goal is to parse as much of the program as possible and report as many errors as possible, not to fix the program.

Error productions. The grammar anticipates common errors with extra rules. With $S ::= A$, $A ::= aA \mid bA \mid a \mid b$ and $B ::= cd$, the input $abcd$ is not in the language, because S does not generate the trailing cd. The augmented grammar adds $E ::= S\,B$. If the parser uses that rule, it has found the error and can report it precisely.

Global correction. The parser looks for the smallest number of insertions, deletions and changes that turn the input into a valid program. For the function `arithmetic_logic` of the first lecture, replacing `a = 3+foo;` by `a = 3;` is a correction of that kind. It gives the best diagnosis and is too expensive to use in practice.

2.2 Top-down parsing

There are two approaches to parsing. A top-down parser starts from the initial symbol and tries to generate a prefix of the input, expanding nonterminals. A bottom-up parser starts from the input as the leaves of the tree and tries to build the tree upward.

Top-down parsing follows a leftmost derivation. For $2 + 3 * 1$ and the grammar without left recursion

```
Exp    ::= Term Exp'
Exp'   ::= + Term Exp' | λ
Term   ::= Factor Term'
Term'  ::= * Factor Term' | λ
Factor ::= ( Exp ) | Num
Num    ::= (0..9)+
```

the parser builds the tree of Figure 6 from the root, expanding the leftmost nonterminal each time. When it has to expand `Term'` after reading 2, the next symbol is $+$, so it chooses $\mathit{Term}' ::= \lambda$; after reading 3 the next symbol is $*$, so it chooses $\mathit{Term}' ::= *\mathit{Factor}\ \mathit{Term}'$. The question in every step is which rule to apply.

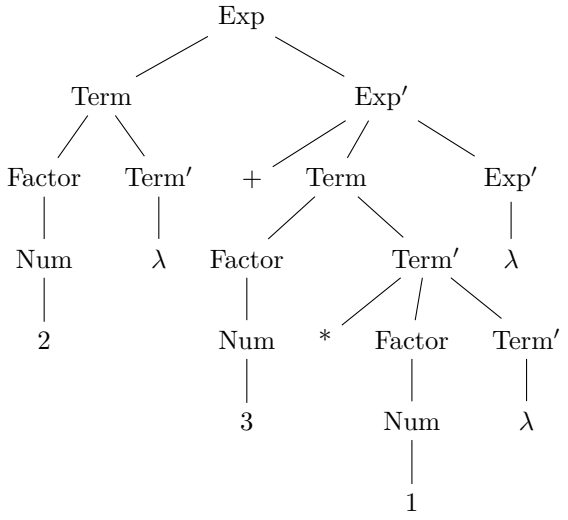


Figure 6: Top-down parse tree of $2 + 3 * 1$.

Recursive descent. The direct implementation gives each nonterminal its own procedure. The lecture's sketch is

```
void A() {
  ProductionRule P = chooseRuleFor(A);
  for (i <- 1..P.length) {
    if (nonTerminals.contain(P[i])) P[i]();
    else if (P[i] == word[j]) j += 1;
    else ParserError();
  }
}
```

If `chooseRuleFor` guesses, the procedure has to backtrack to each decision point when a guess fails. A left-recursive grammar makes it loop forever: `Exp()` would call `Exp()` before reading anything. For the grammar above the choice can be made by looking at the next symbol, and the parser needs no backtracking. In Rascal, with the value of the expression computed along the way:

```
alias Result = tuple[int val, list[str] rest];

Result exp(list[str] input) {
```

```
  <v, rest> = term(input);
  return expPrime(v, rest);
}

Result expPrime(int left, list[str] input) {
  if (["+", *rest] := input) {
    <v, rest2> = term(rest);
    return expPrime(left + v, rest2);
  }
  return <left, input>;
}

Result term(list[str] input) {
  <v, rest> = factor(input);
  return termPrime(v, rest);
}

Result termPrime(int left, list[str] input) {
  if (["*", *rest] := input) {
    <v, rest2> = factor(rest);
    return termPrime(left * v, rest2);
  }
  return <left, input>;
}

Result factor(list[str] input) {
  if (["(", *rest] := input) {
    <v, rest2> = exp(rest);
    if ([")", *rest3] := rest2) return <v, rest3>;
    throw "expected )";
  }
  if ([n, *rest] := input, /^[0-9]+$/ := n) return <toInt(n), rest>;
  throw "unexpected <input == [] ? "end of input" : head(input)>";
}

int evaluate(list[str] tokens) {
  <v, rest> = exp(tokens);
  if (rest != []) throw "unexpected <head(rest)>";
  return v;
}
```

Pattern matching on the list of tokens makes each rule one `if`: the pattern `["+", *rest]` says that the next token is `+`.

2.3 FIRST and FOLLOW

The choice of rule uses two functions of the grammar.

Definition 2.1. For $\alpha \in (N \cup \Sigma)^*$, *first*$(\alpha)$ is the set of terminals that begin the words derived from α, plus λ if $\alpha \Rightarrow^* \lambda$. It is computed with three rules:

- if $X \in \Sigma$, then *first*$(X) = \{X\}$;

- if $X ::= \alpha_1 \alpha_2 \cdots \alpha_k$, then $\sigma \in \mathit{first}(X)$ when $\sigma \in \mathit{first}(\alpha_i)$ for some i and λ belongs to $\mathit{first}(\alpha_j)$ for every $j < i$; if $\lambda \in \mathit{first}(\alpha_j)$ for all j, then $\lambda \in \mathit{first}(X)$;
- if $X ::= \lambda$, then $\lambda \in \mathit{first}(X)$.

Definition 2.2. For a nonterminal A, $\mathit{follow}(A)$ is the set of terminals that can appear right after A in a sentential form. It uses a symbol \$ that marks the end of the input and is not a symbol of the language:

- add \$ to $\mathit{follow}(S)$ for the initial symbol S;
- for a rule $A ::= \alpha B \beta$, every $\sigma \in \mathit{first}(\beta) \setminus \{\lambda\}$ is in $\mathit{follow}(B)$;
- for a rule $A ::= \alpha B$, or $A ::= \alpha B \beta$ with $\lambda \in \mathit{first}(\beta)$, $\mathit{follow}(A) \subseteq \mathit{follow}(B)$.

(The slides write $\alpha \in (N \cup \Sigma)$ for the argument of *first* and X_{i-1} in the second rule; the argument is a string, $(N \cup \Sigma)^*$, and the index refers to α_{i-1}. In the last rule of *follow* the slide repeats $A ::= \alpha B \beta$ where the first case is $A ::= \alpha B$.)

Both are computed by applying the rules until the sets stop changing. Table 3 gives them for the expression grammar. FIRST goes from the bottom of the grammar up: $\mathit{first}(\mathit{Factor})$ is `(` plus the digits, and *Term* and *Exp* start with a *Factor*. FOLLOW goes from the top down: *Exp* is followed by \$ and by the `)` of $\mathit{Factor} ::= (\mathit{Exp})$, and since Exp' can be empty, what follows *Exp* also follows *Term*, together with the + that begins Exp'.

Nonterminal	first	follow
Exp	{ (, 0..9 }	{), \$ }
Exp′	{ +, λ }	{), \$ }
Term	{ (, 0..9 }	{ +,), \$ }
Term′	{ *, λ }	{ +,), \$ }
Factor	{ (, 0..9 }	{ +, *,), \$ }
Num	{ 0..9 }	{ +, *,), \$ }

Table 3: FIRST and FOLLOW for the expression grammar.

In Rascal a grammar is a list of rules, and both functions are fixed points written with `solve`:

```
set[str] firstSeq(Grammar g, map[str, set[str]] f, list[str] alpha) {
  set[str] result = {};
  for (x <- alpha) {
    set[str] fx = isTerminal(g, x) ? {x} : f[x];
    result += fx - {"λ"};
    if ("λ" notin fx) return result;
  }
  return result + {"λ"};
}

map[str, set[str]] first(Grammar g) {
  map[str, set[str]] f = ( a : {} | a <- nonterminals(g) );
  solve (f) {
    for (<a, rhs> <- g) f[a] += firstSeq(g, f, rhs);
  }
  return f;
}
```

```
map[str, set[str]] follow(Grammar g, str s0) {
  map[str, set[str]] f = first(g);
  map[str, set[str]] fo = ( a : {} | a <- nonterminals(g) );
  fo[s0] = {"$"};
  solve (fo) {
    for (<a, rhs> <- g, i <- index(rhs), !isTerminal(g, rhs[i])) {
      set[str] fBeta = firstSeq(g, f, rhs[i + 1 ..]);
      fo[rhs[i]] += fBeta - {"λ"};
      if ("λ" in fBeta) fo[rhs[i]] += fo[a];
    }
  }
  return fo;
}
```

The tests in `LL1.rsc` check the sets of Table 3.

2.4 LL(1) grammars and the parsing table

Definition 2.3. A grammar is **LL(1)** if a predictive parser (recursive descent without backtracking) can parse it reading the input from left to right, following a leftmost derivation and looking at one symbol ahead.

For an LL(1) grammar, the parsing table M stores the decision for each pair of a nonterminal and a lookahead symbol. It is built from FIRST and FOLLOW: for each rule $A ::= \alpha$,

1. for each terminal $\sigma \in \mathit{first}(\alpha)$, add $A ::= \alpha$ to $M[A, \sigma]$;
2. if $\lambda \in \mathit{first}(\alpha)$, add $A ::= \alpha$ to $M[A, \sigma']$ for each terminal $\sigma' \in \mathit{follow}(A)$, and to $M[A, \$]$ if $\$ \in \mathit{follow}(A)$.

(The slide writes $M[A, \alpha]$ in the first step; the column is the terminal σ.) The grammar is LL(1) when no cell has two rules. Table 4 is the table for the expression grammar. The empty cells are errors.

	0..9	+	*	(	)	$
E	T E′			T E′		
E′		+ T E′			λ	λ
T	F T′			F T′		
T′		λ	* F T′		λ	λ
F	Num			(E)		
Num	0..9					

Table 4: LL(1) table for the expression grammar, with E, T and F for Exp, Term and Factor. Each cell shows the right side of the rule for the nonterminal of its row.

```
rel[str, str, Rule] table(Grammar g, str s0) {
  map[str, set[str]] f = first(g);
  map[str, set[str]] fo = follow(g, s0);
  rel[str, str, Rule] m = {};
```

```
  for (r:<a, alpha> <- g) {
    set[str] fa = firstSeq(g, f, alpha);
    for (s <- fa - {"λ"}) m += <a, s, r>;
    if ("λ" in fa) {
      for (s <- fo[a]) m += <a, s, r>;
    }
  }
  return m;
}

bool isLL1(Grammar g, str s0) {
  m = table(g, s0);
  return all(<a, s> <- m<0, 1>, size(m[a, s]) == 1);
}
```

Nonrecursive predictive parsing. The recursion of recursive descent can be replaced by an explicit stack, as in the pushdown automaton built from a grammar in the previous block. The parser keeps the input followed by \$, a stack that starts with $S\,\$$, and the table:

```
void nonRecursiveParser(String w$) {
  a = first(w);
  X = stack.top();
  while (X != $) {
    if (X == a) { stack.pop(); a = next(); }
    else if (terminals.contains(X)) error();
    else if (M[X,a].isEmpty()) error();
    else if (M[X,a] == X ::= Y1 Y2 ... Yk) {
      output(X ::= Y1 Y2 ... Yk);
      stack.pop();
      stack.push(Yk Yk-1 ... Y1);
    }
    X = stack.top();
  }
}
```

The right side is pushed in reverse so that Y_1 ends up on top.

Panic-mode recovery in a predictive parser. Panic mode works as well as the choice of synchronizing symbols, and the parser should find one as soon as possible. The lecture lists good choices:

1. the symbols in *follow*(A): skip tokens until one of them and pop A, and parsing can probably continue;
2. the symbols that begin higher-level constructs (for an expression, the symbols that begin statements);
3. the symbols in *first*(A), to resume parsing A when one appears;
4. if a nonterminal produces λ, use that rule by default;
5. if a terminal on top of the stack does not match, pop it and report it as inserted.

With the first choice, the empty cells of Table 4 in the columns of $follow(A)$ become *synch*: pop A. The other empty cells mean skip the input symbol. Table 5 is the lecture's trace for the input $)4 * +2$. The parser skips the initial), because popping *Exp* there would end the parse with the whole input left, and later pops *Factor* when it finds +, which belongs to the follow set of *Factor*. It reports two errors and reaches the end of the input.

Stack (top first)	Input	Action
Exp \$	$)4 * +2$\$	error, skip)
Exp \$	$4 * +2$\$	Exp ::= Term Exp′
Term Exp′ \$	$4 * +2$\$	Term ::= Factor Term′
Factor Term′ Exp′ \$	$4 * +2$\$	Factor ::= Num, Num ::= 4
4 Term′ Exp′ \$	$4 * +2$\$	match 4
Term′ Exp′ \$	$* + 2$\$	Term′ ::= ∗ Factor Term′
∗ Factor Term′ Exp′ \$	$* + 2$\$	match ∗
Factor Term′ Exp′ \$	$+2$\$	error, $M[Factor, +]$ = synch
Term′ Exp′ \$	$+2$\$	Term′ ::= λ
Exp′ \$	$+2$\$	Exp′ ::= + Term Exp′
+ Term Exp′ \$	$+2$\$	match +
Term Exp′ \$	2\$	Term ::= Factor Term′
Factor Term′ Exp′ \$	2\$	Factor ::= Num, Num ::= 2
2 Term′ Exp′ \$	2\$	match 2
Term′ Exp′ \$	\$	Term′ ::= λ
Exp′ \$	\$	Exp′ ::= λ
\$	\$	end

Table 5: Predictive parsing of $)4 * +2$ with panic-mode recovery.

The parser in Rascal produces this trace step by step (the test `panic` checks the two errors):

```
alias Step = tuple[str stack, str input, str action];

list[Step] parse(Grammar g, str s0, list[str] input) {
  rel[str, str, Rule] m = table(g, s0);
  map[str, set[str]] fo = follow(g, s0);
  list[str] stack = [s0, "$"];
  input += ["$"];
  list[Step] trace = [];
  str show(list[str] xs) = intercalate("", xs);
  while (head(stack) != "$") {
    str x = head(stack);
    str a = head(input);
    str action = "";
    if (x == a) {
      action = "match <a>";
      stack = tail(stack); input = tail(input);
    } else if (isTerminal(g, x)) {
      action = "error, pop <x>";
      stack = tail(stack);
    } else if (m[x, a] == {}) {
      if ((a in fo[x] || a == "$") && tail(stack) != ["$"]) {
        action = "error, M[<x>,<a>] = synch";
        stack = tail(stack);
      } else {
```

```
        action = "error, skip <a>";
        input = tail(input);
      }
    } else {
      Rule r = getOneFrom(m[x, a]);
      action = "<r.lhs> ::= <r.rhs == [] ? "λ" : intercalate(" ", r.rhs)>";
      stack = r.rhs + tail(stack);
    }
    trace += [<show(stack), show(input), action>];
  }
  return trace;
}

list[str] errors(list[Step] trace) = [ s.action | s <- trace, startsWith(s.action, "
    error") ];
bool accepted(Grammar g, str s0, list[str] input) = errors(parse(g, s0, input)) ==
    [];
```

Phrase-level recovery in a predictive parser. The empty cells can also hold calls to error routines that change the input or the stack: `delete()` removes the input symbol, `insert()` inserts a missing symbol and `change()` replaces one. Each routine must make progress (for example, never insert in front of a symbol forever), or the parser can loop.

2.5 Bottom-up parsing

A bottom-up parser reduces the word to the initial symbol. At each step it matches a substring with the right side of a rule and replaces it by the rule's nonterminal. Given a word $\alpha\beta\omega$ and a rule $A ::= \beta$, the reduction gives $\alpha A\omega$, which corresponds to the derivation step $S_0 \Rightarrow^* \alpha A\omega \Rightarrow \alpha\beta\omega$. The choice of rule is not always unique; the parser needs to know when to reduce and which rule to use.

With the grammar with left recursion of the previous block,

```
Exp    ::= Exp + Term | Term
Term   ::= Term * Factor | Factor
Factor ::= ( Exp ) | Num
```

the reductions of $3 * 2$ are

$$3 * 2 \;\rightarrow\; \mathit{Factor} * 2 \;\rightarrow\; \mathit{Term} * 2 \;\rightarrow\; \mathit{Term} * \mathit{Factor} \;\rightarrow\; \mathit{Term} \;\rightarrow\; \mathit{Exp},$$

using $\mathit{Factor} ::= 3$, $\mathit{Term} ::= \mathit{Factor}$, $\mathit{Factor} ::= 2$, $\mathit{Term} ::= \mathit{Term} * \mathit{Factor}$ and $\mathit{Exp} ::= \mathit{Term}$. Read backward, this is the rightmost derivation of $3 * 2$.

Shift-reduce. The parser scans the input from left to right and pushes the symbols on a stack (shift). When the top of the stack matches the right side of a rule, it replaces those symbols by the nonterminal (reduce). It stops when the stack holds only the initial symbol and the input is empty. Table 6 is the trace for $3 * 2$. In the second row the parser could also shift $*$, and after the fourth row it could reduce *Term* to *Exp*; the LR automaton below is what decides.

Stack (top first)	Input	Action
$	$3 * 2$	shift
3 $	$*2$	reduce Factor ::= 3
Factor $	$*2$	reduce Term ::= Factor
Term $	$*2$	shift
$*$ Term $	2	shift
2 $*$ Term $	$	reduce Factor ::= 2
Factor $*$ Term $	$	reduce Term ::= Term $*$ Factor
Term $	$	reduce Exp ::= Term
Exp $	$	accept

Table 6: Shift-reduce parsing of $3 * 2$.

Conflicts. With the grammar

```
Stmt ::= if Cond then Stmt | if Cond then Stmt else Stmt | 0..9
Cond ::= Var Op 0..9
Var  ::= x | y
Op   ::= > | =
```

and the input `if x>0 then 1 else 2`, the parser reaches the configuration with stack `1 then Cond if` and input `else 2$`. It can reduce `1` to `Stmt`, and then it must decide: reduce `if Cond then Stmt` to `Stmt`, or shift `else` to build the longer rule. This is a **shift/reduce conflict**, and it is the dangling `else` of the previous block seen from the parser. Parser generators usually resolve it by shifting, which attaches the `else` to the nearest `if`.

2.6 LR parsing

Definition 2.4. An **LR(k)** parser reads the input from left to right, produces a rightmost derivation in reverse and looks at k symbols ahead to decide each step. A grammar is LR(k) if such a parser exists for it.

The LR parser keeps a stack of states $s_m s_{m-1} \cdots$ instead of symbols. The states come from an automaton that tracks how much of each rule has been seen.

Definition 2.5. An **item** is a rule with a dot in its right side. The rule *Factor* ::= (*Exp*) has four items: $\cdot$(*Exp*) (nothing seen yet), ($\cdot$*Exp*) and (*Exp*$\cdot$) (part seen, can still shift) and (*Exp*)$\cdot$ (everything seen, can reduce).

A state of the automaton is a set of items. Two operations build them. The closure of a set adds, for every item with the dot before a nonterminal B, the items $B ::= \cdot\gamma$ for all rules of B: if the parser expects a B, it also expects the beginning of any of its right sides. $goTo(I, X)$ moves the dot over X in the items of I that have X after the dot, and takes the closure.

```
Set<Item> closure(State I) {
  J = I;
  for (item A ::= α.Bβ in J)
    for (B ::= γ in G)
      if (B ::= .γ not in J) add(B ::= .γ, J);
  return J;
}
```

```
void items(Grammar G') {
  C = { closure({S' ::= .S}) };
  for (I in C)
    for (X in N ∪ Σ)
      if (!isEmpty(goTo(I, X))) add(goTo(I, X), C);
}
```

The grammar is first augmented with a new initial rule $S' ::= S$, so that accepting is reducing that rule. For the expression grammar (with *num* for numbers) the collection has twelve states. Table 7 lists their items and Figure 7 the transitions.

State	Items
I_0	$Exp' ::= \cdot Exp$, $Exp ::= \cdot Exp + Term$, $Exp ::= \cdot Term$, $Term ::= \cdot Term * Factor$, $Term ::= \cdot Factor$, $Factor ::= \cdot(Exp)$, $Factor ::= \cdot num$
I_1	$Exp' ::= Exp\cdot$, $Exp ::= Exp \cdot + Term$
I_2	$Exp ::= Term\cdot$, $Term ::= Term \cdot * Factor$
I_3	$Term ::= Factor\cdot$
I_4	$Factor ::= (\cdot Exp)$, plus the closure items of I_0 for *Exp*, *Term*, *Factor*
I_5	$Factor ::= num\cdot$
I_6	$Exp ::= Exp + \cdot Term$, plus the items for *Term* and *Factor*
I_7	$Term ::= Term * \cdot Factor$, $Factor ::= \cdot(Exp)$, $Factor ::= \cdot num$
I_8	$Factor ::= (Exp\cdot)$, $Exp ::= Exp \cdot + Term$
I_9	$Exp ::= Exp + Term\cdot$, $Term ::= Term \cdot * Factor$
I_{10}	$Term ::= Term * Factor\cdot$
I_{11}	$Factor ::= (Exp)\cdot$

Table 7: The states of the LR automaton for the expression grammar.

The parser reads the next symbol and looks at the current state. If the state has an item with that terminal after the dot, it shifts and moves to the target of the transition. If the state has a complete item $A ::= \beta\cdot$ and the next symbol is in $follow(A)$, it reduces: it pops one state per symbol of β and follows the transition on A from the state now on top. This is the SLR method (simple LR). For $3 * 2$: in I_0 the parser shifts *num* to I_5; in I_5 the next symbol $*$ is in $follow(Factor)$, so it reduces $Factor ::= num$ and goes from I_0 to I_3; in I_3 it reduces $Term ::= Factor$ and goes to I_2; in I_2 the item $Term ::= Term \cdot * Factor$ says to shift $*$, while the item $Exp ::= Term\cdot$ would reduce only on $+$,) or \$. That is how the automaton resolves the choice of the shift-reduce trace.

In Rascal, items, closure and `goTo` are a few lines each:

```
alias Item = tuple[Rule rule, int dot];
alias State = set[Item];

str afterDot(Item i) = i.dot < size(i.rule.rhs) ? i.rule.rhs[i.dot] : "";

State closure(Grammar g, State items) {
  solve (items) {
    for (i <- items, b := afterDot(i), b in nonterminals(g), r:<b, _> <- g) {
      items += {<r, 0>};
    }
  }
  return items;
}
```

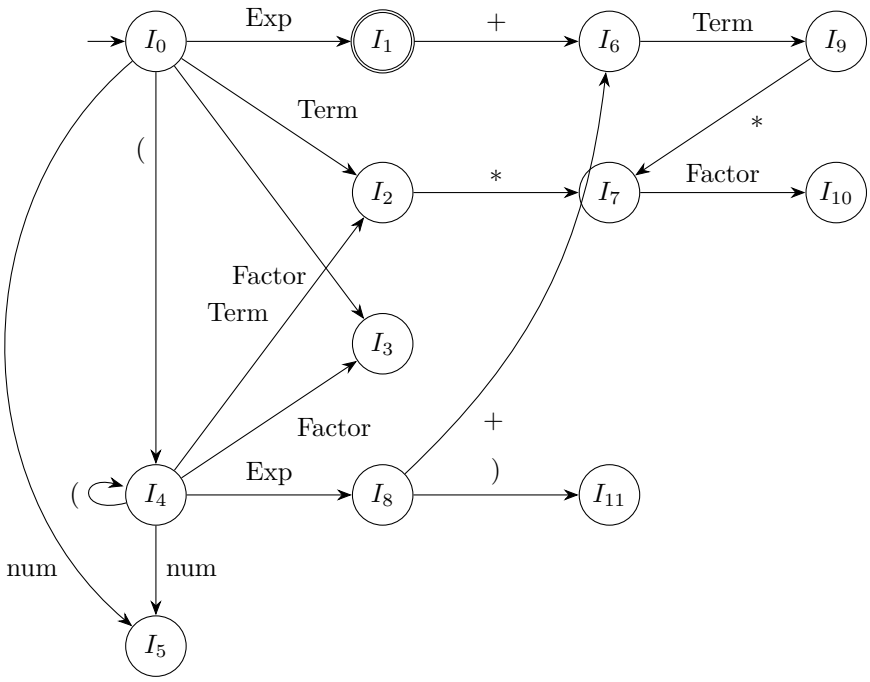


Figure 7: Transitions of the LR automaton. Not drawn: from I_6 and I_7 on (to I_4 and on *num* to I_5, and from I_6 on *Factor* to I_3.

```
State goTo(Grammar g, State items, str x)
  = closure(g, { <i.rule, i.dot + 1> | i <- items, afterDot(i) == x });

list[State] collection(Grammar g) {
  list[State] states = [closure(g, {<g[0], 0>})];
  set[str] symbols = { x | r <- g, x <- r.rhs };
  solve (states) {
    for (s <- states, x <- symbols) {
      State next = goTo(g, s, x);
      if (next != {} && next notin states) states += [next];
    }
  }
  return states;
}
```

and the SLR table and the parser:

```
rel[int, str, str] actions(Grammar g) {
  list[State] states = collection(g);
  map[str, set[str]] fo = follow(g, g[0].lhs);
  rel[int, str, str] act = {};
  for (k <- index(states), i <- states[k]) {
    str x = afterDot(i);
    if (x != "" && isTerminal(g, x)) {
      act += <k, x, "shift <indexOf(states, goTo(g, states[k], x))>">;
    } else if (x == "" && i.rule == g[0]) {
      act += <k, "$", "accept">;
```

```
    } else if (x == "") {
      for (a <- fo[i.rule.lhs]) act += <k, a, "reduce <indexOf(g, i.rule)>">;
    }
  }
  return act;
}

bool hasConflict(Grammar g) = any(<k, a> <- actions(g)<0, 1>, size(actions(g)[k, a])
    > 1);

list[str] shiftReduce(Grammar g, list[str] input) {
  list[State] states = collection(g);
  rel[int, str, str] act = actions(g);
  list[int] stack = [0];
  list[str] symbols = [];
  input += ["$"];
  list[str] trace = [];
  while (true) {
    set[str] options = act[head(stack), head(input)];
    if (size(options) != 1) return trace + ["error"];
    str a = getOneFrom(options);
    if (a == "accept") return trace + ["accept"];
    if (/shift <n:[0-9]+>/ := a) {
      trace += ["shift <head(input)>"];
      symbols = [head(input)] + symbols;
      stack = [toInt(n)] + stack;
      input = tail(input);
    } else if (/reduce <n:[0-9]+>/ := a) {
      Rule r = g[toInt(n)];
      trace += ["reduce <r.lhs> ::= <intercalate(" ", r.rhs)>"];
      int k = size(r.rhs);
      stack = stack[k..];
      symbols = [r.lhs] + symbols[k..];
      State next = goTo(g, states[head(stack)], r.lhs);
      stack = [indexOf(states, next)] + stack;
    }
  }
}
```

The tests check that the collection has twelve states, that the table has no conflicts, that the trace of $3 * 2$ is the one of Table 6 and that the grammar of the `if` statement has a conflict on `else`.

2.7 Precedence climbing

This subsection goes beyond the slides. Writing one nonterminal per precedence level becomes tedious when a language has many operators. Precedence climbing, or Pratt parsing, uses a table of binding powers instead. It parses an atom, then keeps reading operators while their precedence is at least the current minimum, parsing each right operand with a higher minimum so that operators of the same level associate to the left:

```
PRECEDENCE = {"+": 1, "-": 1, "*": 2, "/": 2}

def parse_expr(tokens, min_bp=0):
```

```
    left = parse_atom(tokens)
    while True:
        op = tokens.peek()
        if op not in PRECEDENCE or PRECEDENCE[op] < min_bp:
            break
        tokens.advance()
        right = parse_expr(tokens, PRECEDENCE[op] + 1)
        left = (op, left, right)
    return left

def parse_atom(tokens):
    tok = tokens.advance()
    if tok == "(":
        e = parse_expr(tokens, 0)
        tokens.expect(")")
        return e
    return int(tok)
```

For $1 + 2 * 3$ the call for the right operand of $+$ has minimum 2, so it takes $2 * 3$ whole before returning, and the result is $1 + (2 * 3)$. Adding a level is one entry in the table. Nystrom's book builds its expression parser this way.

2.8 Exercises

Exercise 2.6. Compute FIRST and FOLLOW for the grammar $S ::= aB \mid bA \mid \lambda$, $A ::= aS \mid bAA$, $B ::= bS \mid aBB$ of the previous block. Is it LL(1)?

Solution. $\mathit{first}(S) = \{a, b, \lambda\}$, $\mathit{first}(A) = \{a, b\}$, $\mathit{first}(B) = \{a, b\}$. $\mathit{follow}(S) = \{\$\} \cup \mathit{follow}(A) \cup \mathit{follow}(B)$, because S ends the rules $A ::= aS$ and $B ::= bS$; $\mathit{follow}(A)$ contains $\mathit{follow}(S)$ and $\mathit{first}(A) = \{a, b\}$ (from bAA), and the same for B. So every FOLLOW set is $\{a, b, \$\}$. The grammar is not LL(1): $M[S, a]$ holds $S ::= aB$ (from FIRST) and $S ::= \lambda$ (because $a \in \mathit{follow}(S)$).

Exercise 2.7. Run the predictive parser of Table 4 on $(1 + 2) * 3$ and write the rules it outputs.

Solution. Exp ::= Term Exp′, Term ::= Factor Term′, Factor ::= (Exp), Exp ::= Term Exp′, Term ::= Factor Term′, Factor ::= Num, Num ::= 1, Term′ ::= λ, Exp′ ::= + Term Exp′, Term ::= Factor Term′, Factor ::= Num, Num ::= 2, Term′ ::= λ, Exp′ ::= λ, Term′ ::= $*$ Factor Term′, Factor ::= Num, Num ::= 3, Term′ ::= λ, Exp′ ::= λ. This is the leftmost derivation of the word, rule by rule.

Exercise 2.8. Write the shift-reduce trace for `if x>0 then 1 else 2` up to the conflict, and say which action resolves it the usual way.

Solution. Shift `if`, shift `x`, reduce `Var ::= x`, shift `>`, reduce `Op ::= >`, shift `0`, reduce `Cond ::= Var Op 0`, shift `then`, shift `1`, reduce `Stmt ::= 1`. Now the stack is `Stmt then Cond if` and the next symbol is `else`: shift or reduce `Stmt ::= if Cond then Stmt`. Shifting `else` gives the usual meaning.

Exercise 2.9. Compute $\mathit{goTo}(I_0, \texttt{(})$ for the expression grammar and check that it is I_4.

Solution. The only item of I_0 with `(` after the dot is *Factor* ::= ·(*Exp*). Moving the dot gives *Factor* ::= (·*Exp*). The closure adds the items for *Exp* (dot before *Exp*), then for *Term* and *Factor*, which are the same closure items as in I_0, without *Exp*′ ::= ·*Exp*.

3 Abstract syntax trees and attribute grammars

Some rules of a language cannot be captured by its syntax definition: type consistency, or the use of `return` statements. Semantic analysis checks them, and it works on a tree that is simpler than the parse tree. This section and the next follow Wilhelm, Seidl and Hack [5, Ch. 4] and Scott and Aldrich [4, Ch. 4].

3.1 Abstract syntax trees

Definition 3.1. An **abstract syntax tree** (AST) is a parse tree with the nonessential information removed: nonterminal nodes are replaced by nodes for the constructs of the word they represent, and unit productions (rules such as *Exp* ::= *Term*) are collapsed.

Figure 8 compares the two trees for `if x then y else z` in a grammar where the condition is an expression and the branches are statements. The parse tree has a node for every rule and every keyword; the AST has one node for the conditional, with three children. ASTs are what compilers use to generate code and what interpreters run.

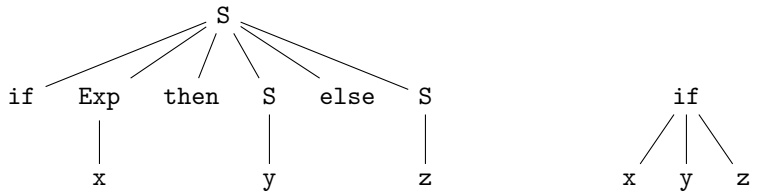


Figure 8: Parse tree (left) and AST (right) of a conditional.

Example. For the imperative language of the semantics block, with assignment $l := e$, sequence, conditional, `while` and dereferencing $!l$, the program

$$y := 0 \ ; \ \textbf{if } !x = !y \ \textbf{then } x := !x + 1 \ ; \ y := !y - 1 \ \textbf{else } x := !x - 1 \ ; \ y := !y + 1$$

has the AST of Figure 9. The top-level semicolon is the root. $!x$ is a node DEREF applied to the location x: the location names a cell of the memory, and dereferencing reads its content.

3.2 Attribute grammars

Definition 3.2. An **attribute grammar** associates attributes with the symbols of a grammar and, with each rule, semantic functions that compute the attributes of the symbols of the rule. The attributes give the grammar elements a value or a meaning.

For the expression grammar, an attribute *val* holds the value of each expression. When a nonterminal appears twice in a rule, subscripts tell the occurrences apart:

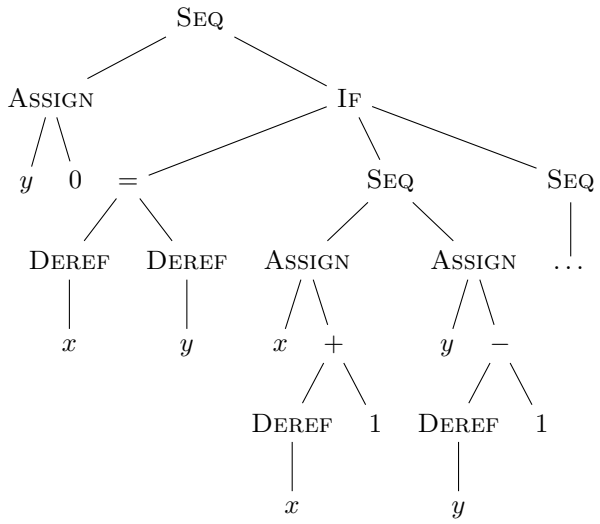


Figure 9: AST of the example program. The `else` branch is symmetric to the `then` branch.

Rule	Semantic function
$Exp_1 ::= Exp_2 + Term$	$Exp_1.val := sum(Exp_2.val, Term.val)$
$Exp_1 ::= Exp_2 - Term$	$Exp_1.val := difference(Exp_2.val, Term.val)$
$Exp ::= Term$	$Exp.val := Term.val$
$Term_1 ::= Term_2 * Factor$	$Term_1.val := product(Term_2.val, Factor.val)$
$Term_1 ::= Term_2/Factor$	$Term_1.val := quotient(Term_2.val, Factor.val)$
$Term ::= Factor$	$Term.val := Factor.val$
$Factor ::= (Exp)$	$Factor.val := Exp.val$
$Factor_1 ::= -Factor_2$	$Factor_1.val := add_inverse(Factor_2.val)$
$Factor ::= Num$	$Factor.val := Num.val$

Synthesized attributes. Decorating (annotating) a parse tree starts at the leaves and propagates up the tree following the semantic functions. An attribute computed from the children of a node is **synthesized**. Figure 10 shows the decoration of $7 + 3 * 4$: the leaves get the values of the numbers, the product node gets 12, and the root gets $7 + 12 = 19$.

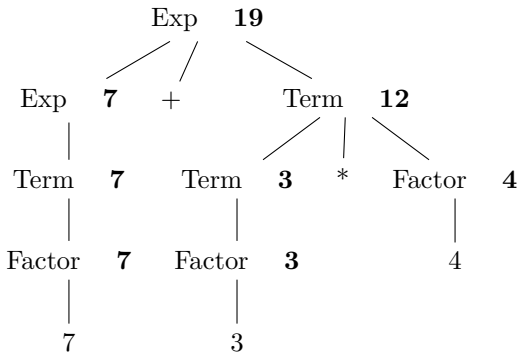


Figure 10: Parse tree of $7 + 3 * 4$ decorated with the synthesized attribute *val*.

In Rascal, each semantic function is one equation over a concrete syntax pattern, and the attribute is the function:

```
int val((Exp) `<Exp e> + <Term t>`)      = val(e) + val(t);
int val((Exp) `<Exp e> - <Term t>`)      = val(e) - val(t);
int val((Exp) `<Term t>`)                = val(t);
int val((Term) `<Term t> * <Factor f>`)  = val(t) * val(f);
int val((Term) `<Term t> / <Factor f>`)  = val(t) / val(f);
int val((Term) `<Factor f>`)             = val(f);
int val((Factor) `( <Exp e> )`)          = val(e);
int val((Factor) `- <Factor f>`)         = -val(f);
int val((Factor) `<Num n>`)              = toInt("<n>");
```

Inherited attributes. An attribute computed from the parent or the siblings of a node is **inherited**. Inherited attributes carry values from the left part of an expression into the right part, for example to preserve associativity in a grammar without left recursion. With

Rule	Semantic functions
$Exp ::= Num\ ExpT$	$ExpT.inh := Num.val;\ \ Exp.val := ExpT.syn$
$ExpT_1 ::= -Num\ ExpT_2$	$ExpT_2.inh := ExpT_1.inh - Num.val;\ \ ExpT_1.syn := ExpT_2.syn$
$ExpT ::= \lambda$	$ExpT.syn := ExpT.inh$

the value of $9-4-3$ is computed as in Figure 11. The 9 goes down into the first $ExpT$; there $9-4=5$ goes down into the second; there $5-3=2$ goes down into the empty $ExpT$, which copies its inherited value into its synthesized one; and the 2 goes back up to the root. The subtraction associates to the left, $(9-4)-3=2$, even though the tree grows to the right. Computing $9-(4-3)=8$ would be the error of ignoring associativity.

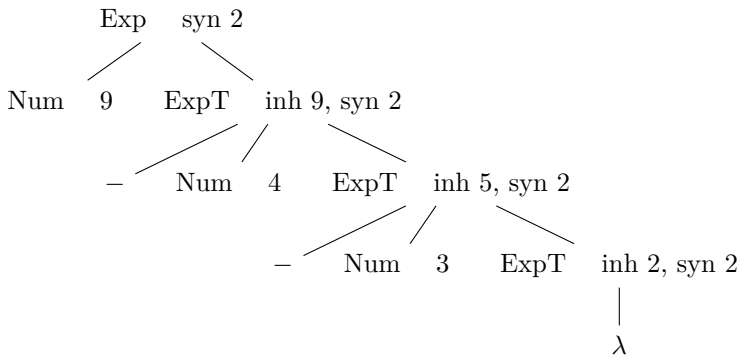


Figure 11: Decoration of $9-4-3$ with an inherited attribute (*inh*) and a synthesized one (*syn*).

In Rascal the inherited attribute is an extra parameter:

```
int val((ExpI) `<Num n> <ExpT t>`) = syn(t, toInt("<n>"));

int syn((ExpT) `- <Num n> <ExpT t>`, int inh) = syn(t, inh - toInt("<n>"));
default int syn(ExpT _, int inh) = inh;
```

3.3 Building ASTs

Attribute grammars can also build the AST from the parse tree. An attribute *ptr* points to the AST node of each grammar symbol, and three constructors create nodes: *make_leaf*(n),

make_bin_op(op, l, r) and *make_un_op*(op, a).

Bottom-up. With the grammar with left recursion, *ptr* is synthesized:

Rule	Semantic function
$Exp_1 ::= Exp_2 + Term$	$Exp_1.ptr :=$ *make_bin_op*$(\text{“}+\text{”}, Exp_2.ptr, Term.ptr)$
$Exp_1 ::= Exp_2 - Term$	$Exp_1.ptr :=$ *make_bin_op*$(\text{“}-\text{”}, Exp_2.ptr, Term.ptr)$
$Exp ::= Term$	$Exp.ptr := Term.ptr$
$Term_1 ::= Term_2 * Fact$	$Term_1.ptr :=$ *make_bin_op*$(\text{“}*\text{”}, Term_2.ptr, Fact.ptr)$
$Term_1 ::= Term_2/Fact$	$Term_1.ptr :=$ *make_bin_op*$(\text{“}/\text{”}, Term_2.ptr, Fact.ptr)$
$Term ::= Fact$	$Term.ptr := Fact.ptr$
$Fact ::= (Exp)$	$Fact.ptr := Exp.ptr$
$Fact_1 ::= -Fact_2$	$Fact_1.ptr :=$ *make_un_op*$(\text{“}-\text{”}, Fact_2.ptr)$
$Fact ::= Num$	$Fact.ptr :=$ *make_leaf*$(Num.val)$

The slide assigns $Exp_1.val$ in the rule for subtraction; the attribute is *ptr*, since the rule builds a node. It also writes the unary rule with *Factor* on the left and *Fact* on the right; the table uses *Fact* throughout.

For $(1 + 2) * 4$ the leaves become nodes first, then the sum, then the product (Figure 12). The parentheses and the unit rules disappear: they only pass pointers up.

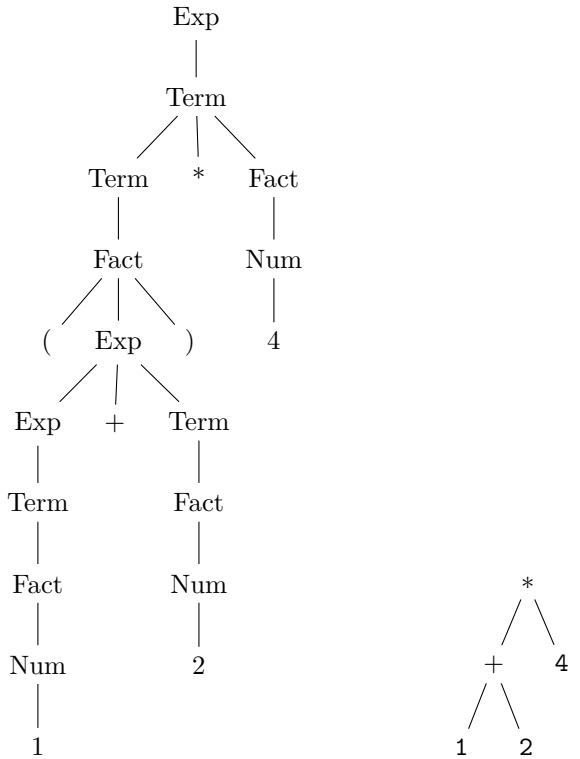


Figure 12: Parse tree of $(1 + 2) * 4$ and the AST its *ptr* attributes build. The + node is created when the parser reduces $Exp ::= Exp + Term$ inside the parentheses, the $*$ node at $Term ::= Term * Fact$.

```
data AST = leaf(int n) | binOp(str op, AST lhs, AST rhs) | unOp(str op, AST arg);
```

```
AST makeLeaf(int n) = leaf(n);
AST makeBinOp(str op, AST l, AST r) = binOp(op, l, r);
AST makeUnOp(str op, AST a) = unOp(op, a);
```

```
AST ptr((Exp) `<Exp e> + <Term t>`)      = makeBinOp("+", ptr(e), ptr(t));
AST ptr((Exp) `<Exp e> - <Term t>`)      = makeBinOp("-", ptr(e), ptr(t));
AST ptr((Exp) `<Term t>`)                = ptr(t);
AST ptr((Term) `<Term t> * <Factor f>`)  = makeBinOp("*", ptr(t), ptr(f));
AST ptr((Term) `<Term t> / <Factor f>`)  = makeBinOp("/", ptr(t), ptr(f));
AST ptr((Term) `<Factor f>`)             = ptr(f);
AST ptr((Factor) `( <Exp e> )`)          = ptr(e);
AST ptr((Factor) `- <Factor f>`)         = makeUnOp("-", ptr(f));
AST ptr((Factor) `<Num n>`)              = makeLeaf(toInt("<n>"));
```

Top-down. With the grammar without left recursion the operator and its left operand are in different subtrees, so the tree built so far has to travel down to the right as an inherited attribute *st*:

Rule	Semantic functions
$Exp ::= Term\ Terms$	$Terms.st := Term.ptr$; $Exp.ptr := Terms.ptr$
$Terms_1 ::= +Term\ Terms_2$	$Terms_2.st := make_bin_op(" + ", Terms_1.st, Term.ptr)$; $Terms_1.ptr := Terms_2.ptr$
$Terms_1 ::= -Term\ Terms_2$	same, with " − "
$Terms ::= \lambda$	$Terms.ptr := Terms.st$
$Term ::= Fact\ FactTerm$	$FactTerm.st := Fact.ptr$; $Term.ptr := FactTerm.ptr$
$FactTerm_1 ::= *Fact\ FactTerm_2$	$FactTerm_2.st := make_bin_op(" * ", FactTerm_1.st, Fact.ptr)$; $FactTerm_1.ptr := FactTerm_2.ptr$
$FactTerm_1 ::= /Fact\ FactTerm_2$	same, with "/"
$FactTerm ::= \lambda$	$FactTerm.ptr := FactTerm.st$
$Fact_1 ::= -Fact_2$	$Fact_1.ptr := make_un_op(" - ", Fact_2.ptr)$
$Fact ::= (Exp)$	$Fact.ptr := Exp.ptr$
$Fact ::= Num$	$Fact.ptr := make_leaf(Num.val)$

Four rules on the slide differ from this table. In the first rule it assigns $Terms.ptr := Term.ptr$, but the left operand has to go into the inherited *st*, as in the other rules of *Terms*. The same happens with *FactTerm.st* (written *FafctTerm.ptr*). The rules for $*$ and / compute $FactTerm_1.st$ from *Fact.st* and $FactTerm_2.ptr$, the reverse of the rules for + and −; the node must be built from the inherited tree and the new factor and passed down to $FactTerm_2$. And the parenthesis rule is written $Fact ::= Exp$.

```
AST ptr((ExpTD) `<TermTD t> <Terms ts>`) = ptrTerms(ts, ptr(t));

AST ptrTerms((Terms) `+ <TermTD t> <Terms ts>`, AST st) = ptrTerms(ts, makeBinOp("+",
     st, ptr(t)));
AST ptrTerms((Terms) `- <TermTD t> <Terms ts>`, AST st) = ptrTerms(ts, makeBinOp("-",
     st, ptr(t)));
default AST ptrTerms(Terms _, AST st) = st;

AST ptr((TermTD) `<Fact f> <FactTerm ft>`) = ptrFT(ft, ptr(f));
```

```
AST ptrFT((FactTerm) `* <Fact f> <FactTerm ft>`, AST st) = ptrFT(ft, makeBinOp("*",
    st, ptr(f)));
AST ptrFT((FactTerm) `/ <Fact f> <FactTerm ft>`, AST st) = ptrFT(ft, makeBinOp("/",
    st, ptr(f)));
default AST ptrFT(FactTerm _, AST st) = st;

AST ptr((Fact) `( <ExpTD e> )`) = ptr(e);
AST ptr((Fact) `<Num n>`)       = makeLeaf(toInt("<n>"));
```

The tests check that both constructions build the same AST for $(1+2)*4$ and for $1-2*3+4/2$, and that $9-4-3$ becomes $(9-4)-3$.

3.4 Exercises

Exercise 3.3. Decorate the parse tree of $8/2-(1+1)$ with the synthesized attribute *val*.

Solution. The leaves get 8, 2, 1 and 1. *Term* ::= *Term*/*Factor* gives $8/2=4$. Inside the parentheses, *Exp* ::= *Exp* + *Term* gives 2, and *Factor* ::= (*Exp*) copies it. The root, *Exp* ::= *Exp* − *Term*, gives $4-2=2$.

Exercise 3.4. Extend the inherited-attribute grammar for $9-4-3$ with addition, and compute $10-3+2$.

Solution. Add the rule $ExpT_1 ::= +Num\ ExpT_2$ with

$$ExpT_2.inh := ExpT_1.inh + Num.val \qquad ExpT_1.syn := ExpT_2.syn.$$

The inherited values are 10, then $10-3=7$, then $7+2=9$, and 9 goes up to the root.

Exercise 3.5. Build the AST of **while** $!i < !n$ **do** $s := !s + !i\,;\ i := !i + 1$.

Solution. The root is WHILE with two children: the condition $<(\text{DEREF}(i), \text{DEREF}(n))$ and the body $\text{SEQ}(\text{ASSIGN}(s, +(\text{DEREF}(s), \text{DEREF}(i))), \text{ASSIGN}(i, +(\text{DEREF}(i), 1)))$.

Exercise 3.6. Run the top-down construction on $6/3/2$ and show the value of *st* in each *FactTerm*.

Solution. The first *FactTerm* inherits the leaf 6. It reads /3 and passes $st = /(6,3)$ to the second, which reads /2 and passes $/(/(6,3),2)$ to the empty *FactTerm*. That tree goes back up as *ptr*: the division associates to the left, $(6/3)/2=1$.

4 Semantic analysis

Semantic analysis checks the rules that the grammar cannot express. Some checks happen when the program is compiled (static analysis) and others when it runs (dynamic analysis).

4.1 Static analysis

Static analysis enforces semantic rules at compile time. Violating one of these rules usually produces a compile error. Typical rules are the visibility of variables, the checks on identifiers and other context conditions, such as the number and types of the arguments of a call.

Validity and visibility rules. When the compiler finds a variable, as in `x = x + 1;`, it has to determine which declaration the name refers to and whether that declaration is accessible at that point. In

```
... {
  x = 5;
  x;
}
```

the question is whether a declaration of `x` is visible inside the block. The lecture's harder case is a loop inside a loop with the same variable:

```
    for (int i = 0; i < 10; i++) {
        for (int i = 0; i < 20; i++) {
            last = i;
            lines++;
        }
    }
```

In C and C++ each `for` opens a block, and the inner `int i` declares a new variable that hides the outer one inside the inner loop. The program prints $0, 1, \ldots, 19$ ten times: 200 lines. The file `nested_for.c` checks the count. In Java the same code does not compile: `variable i is already defined in method main(String[])`. Java forbids a local variable from hiding another local variable of the same method. The syntax is the same in both languages; the visibility rules are different.

Symbol tables chained by blocks. The compiler keeps one table per block and chains each table to the table of the enclosing block. In the lecture's C example

```
int a; int b;
void p(); void q();
void p()
{ int a; int c; }
int c; int d;
void q()
{ int a; int d;
  void r()
  { int a; int c; }
}
```

the blocks are numbered in the order they open: block 1 is the global scope, block 2 the body of `p`, block 3 the body of `q` and block 4 the body of `r`. At the point inside block 4 the chain is the one of Figure 13. Block 2 is not in it: it was closed before block 3 started. A name is searched from the innermost table outward, so `a` refers to the declaration in block 4, `d` to the one in block 3 and `b` to the global one.

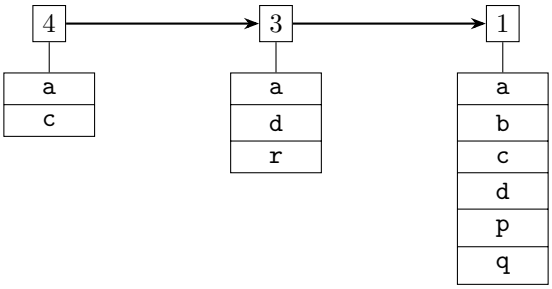


Figure 13: The chain of symbol tables inside block 4.

4.2 Identification of identifiers

Definition 4.1. The **identification of identifiers** links each applied occurrence of an identifier (a use) to a defining occurrence (its declaration), or finds that no such link exists or that more than one exists.

Identification is based on six operations on the symbol table (Table 8).

Operation	Meaning
`create_table()`	creates an empty symbol table
`enter_block()`	registers the opening of a new block
`exit_block()`	resets the table to its state before the last `enter_block`
`enter_id(id, ref)`	enters `id` in the current block, with the reference `ref` to its declaration node
`search_id(id)`	searches the defining occurrence of `id` in all the open blocks, innermost first; returns the reference or null
`search_block_id(id)`	searches `id` only in the current block; returns the reference or null

Table 8: Operations on the symbol table.

In Rascal, with the table as a list of maps (the head is the current block):

```
alias Table = list[map[str, int]];

Table createTable()                          = [()];
Table enterBlock(Table t)                    = [()] + t;
Table exitBlock(Table t)                     = tail(t);
Table enterId(Table t, str id, int ref)      = [head(t) + (id: ref)] + tail(t);

int searchId(Table t, str id) {
  for (block <- t, id in block) return block[id];
  return -1;
}
int searchBlockId(Table t, str id) = id in head(t) ? head(t)[id] : -1;
```

The lecture's language is

```
layout Spaces = [\ \t\n]* !>> [\ \t\n];
keyword Reserved = "int";
lexical Id  = ([a-z][a-z0-9]* !>> [a-z0-9]) \ Reserved;
lexical Num = [0-9]+ !>> [0-9];
```

```
start syntax Block = Item*;
syntax Item = Type Id ";" | Id "=" E ";" | "{" Block "}";
syntax Type = "int";
syntax E    = Id | Num;
```

and the word is `int a; { a = 4; int bar; }`. Its parse tree is in Figure 14. In the attribute grammar of the lecture each identifier leaf is a record $\langle identifier, name, ref, succs\rangle$ with the concrete name, a reference to its declaration and the list of successors, and each interior node is $\langle succs, rhs\rangle$, its children and the right side of the rule that produced it. The lexer gives the identifier tokens their position: `<Id, bar, (1,c)>`.

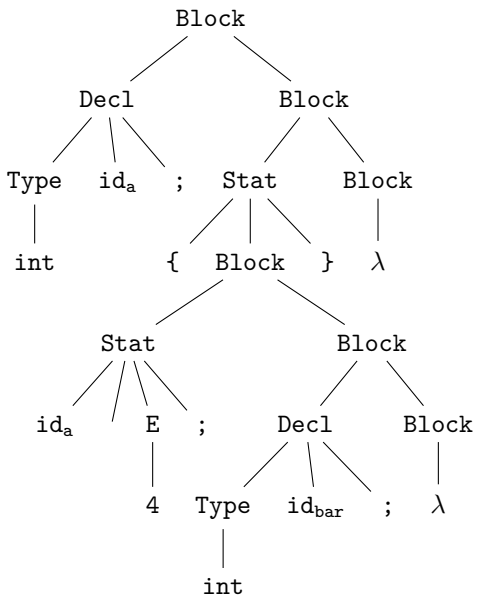


Figure 14: Parse tree of `int a; { a = 4; int bar; }`.

Each node has a procedure `process()` that does its part of the identification, according to the rule of the node:

```
process() {
  Decl ref;
  switch (rhs) {
    case "Type id;":
      ref := table.search_block_id(succs[1].id);
      if (ref != null) error();
      else table.enter_id(succs[1].id, this);
      return;
    case "id = E;":
      ref := table.search_id(succs[0].id);
      if (ref == null) error();
      else succs[0].ref := ref;
      return;
    case "{ Block }":
      table.enter_block();
      succs[1].process();
      table.exit_block();
```

```
      return;
    }
  }
```

A declaration uses `search_block_id`: a name may be declared again in an inner block (it hides the outer one), but not twice in the same block. A use must use `search_id`, because the declaration may be in any enclosing block. On slide 37 the case `"id=E;"` uses `search_block_id`; with that version the use of `a` inside the block finds nothing, because `a` is declared in the outer block, and the correct program is reported as wrong. Table 9 follows the corrected version on the lecture's word.

Node processed	Operation	Table after (current block first)
start	`create_table()`	[{}]
`int a;`	`search_block_id(a)` = null, `enter_id(a, 1)`	[{a↦1}]
`{`	`enter_block()`	[{}, {a↦1}]
`a = 4;`	`search_id(a)` = 1, so `a.ref := 1`	[{}, {a↦1}]
`int bar;`	`search_block_id(bar)` = null, `enter_id(bar, 2)`	[{bar↦2}, {a↦1}]
`}`	`exit_block()`	[{a↦1}]

Table 9: Identification on `int a; { a = 4; int bar; }`.

In Rascal, `process` walks the items of each block. The option `slideVersion` switches the search of uses to `search_block_id`, and the test `slideError` shows the false error it produces:

```
tuple[list[str] errors, list[tuple[str, int]] uses] process(str program, bool
    slideVersion = false) {
  Table table = createTable();
  list[str] errors = [];
  list[tuple[str, int]] uses = [];
  int declarations = 0;

  void use(str x) {
    int ref = slideVersion ? searchBlockId(table, x) : searchId(table, x);
    if (ref == -1) errors += ["<x> is not declared"];
    else uses += [<x, ref>];
  }

  void item(Item i) {
    switch (i) {
      case (Item) `<Type _> <Id x>;`: {
        if (searchBlockId(table, "<x>") != -1) {
          errors += ["<x> is declared twice in the same block"];
        } else {
          declarations += 1;
          table = enterId(table, "<x>", declarations);
        }
      }
      case (Item) `<Id x> = <E e>;`: {
        use("<x>");
        if ((E) `<Id y>` := e) use("<y>");
      }
      case (Item) `{ <Block b> }`: {
        table = enterBlock(table);
        for (j <- items(b)) item(j);
```

```
        table = exitBlock(table);
      }
    }
  }

  for (i <- items(parse(#start[Block], program).top)) item(i);
  return <errors, uses>;
}
```

Decorated ASTs. This paragraph goes beyond the slides. The result of semantic analysis is the same tree with more information: the `ref` of each use, the type of each expression, the kind of each operator (integer or floating-point addition). Later phases read these annotations instead of computing them again.

4.3 Dynamic analysis

Some rules can only be checked while the program runs: division by zero, array indices out of bounds. Dynamic analysis is code that the compiler generates, or that the programmer writes, to enforce them at run time. The lecture names three forms.

Assertions. A condition checked at a point of the program:

```
int divide(int numerator, int denominator) {
    assert(denominator != 0);
    return numerator / denominator;
}
```

If the condition is false the program stops with a message that says where.

Invariants. A condition that must hold every time the program passes through a point, for example at the start of each iteration of a loop. For a loop that adds the first n numbers, $s = 0+1+\cdots+(i-1)$ holds at the start of every iteration.

Pre- and postconditions. Conditions that must hold before and after a piece of code. In the lecture's example

$$\{x \leq 10\} \ \texttt{while (x < 10) x = x + 1;} \ \{x = 10\}$$

if $x \leq 10$ before the loop, then $x = 10$ after it. The semantics block returns to this notation (Hoare triples), where these conditions are proven instead of checked.

4.4 More static analyses

This subsection goes beyond the slides.

Kinds of semantic errors. Besides undeclared names and names declared twice in a block, which the identification above detects, static analysis reports arity mismatches (a call with a different number of arguments than the declaration) and type mismatches (`3 + true`). The lookup of a name can be written as two rules over the symbol table Γ:

$$\frac{(x : \tau) \in \Gamma}{\Gamma \vdash x \Rightarrow \tau}\ \textsc{Found} \qquad\qquad \frac{x \notin \mathrm{dom}(\Gamma)}{\Gamma \vdash x \Rightarrow \textsc{undeclared}}\ \textsc{Undeclared}$$

When Γ is a chain $\Gamma_{inner} :: \Gamma_{outer}$, the search looks at Γ_{inner} first, which gives shadowing.

Data-flow analysis. Other analyses follow how values move through the program. A definition (an assignment) *reaches* a point if some path goes from the definition to the point without reassigning the variable. In

```
x = 1;
y = x;
x = 2;
print(x);
```

`x = 1` reaches `y = x` but not `print(x)`. If no definition of a variable reaches a use, the use reads an uninitialized variable. A variable is *live* at a point if its current value may be read later before being overwritten; a variable that is never live after an assignment was assigned for nothing. Both analyses propagate facts along the control-flow graph until nothing changes, like FIRST and FOLLOW.

4.5 Exercises

Exercise 4.2. What does the nested `for` of this section print in C? What happens in Java? Change the Java version so that it prints the same as the C one.

Solution. In C, the numbers 0 to 19, ten times. Java rejects the program because the inner `i` would hide a local variable of the same method. Renaming the inner variable to `j` and printing `j` gives the same output.

Exercise 4.3. For the C program with blocks 1 to 4, give the chain of symbol tables at a point inside block 2, and say which declaration each of `a`, `b`, `c` and `d` refers to there.

Solution. The chain is block 2 $\rightarrow$ block 1. At that point block 1 only contains `a`, `b`, `p` and `q`, because `c` and `d` are declared later in the text. So `a` and `c` refer to block 2, `b` to block 1, and `d` is not declared yet: using it there is an error.

Exercise 4.4. Run `process()` on `int a; { int a; a = 1; } a = 2; b = a;`. Which declaration does each use of `a` refer to, and which errors are reported?

Solution. The inner `int a;` is allowed: `search_block_id` only looks at the inner block. The use `a = 1` finds declaration 2. After `exit_block()`, `a = 2` finds declaration 1. In `b = a`, `search_id(b)` returns null and an error is reported; `a` refers to declaration 1.

Exercise 4.5. In the program `x = 1; y = 2; if (y > 0) x = 3; print(x);`, which definitions of `x` reach `print(x)`?

Solution. Both. If the condition is false, `x = 1` reaches the `print`; if it is true, `x = 3` does. A static analysis does not know the value of `y` in general, so it keeps both.

References

[1] Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools*. Addison-Wesley, 2nd edition, 2007.

[2] Kent D. Lee. *Foundations of Programming Languages*. Springer, 2nd edition, 2017.

[3] Robert Nystrom. *Crafting Interpreters*. Genever Benning, 2021.

[4] Michael L. Scott and Jonathan Aldrich. *Programming Language Pragmatics*. Morgan Kaufmann, 5th edition, 2025.

[5] Reinhard Wilhelm, Helmut Seidl, and Sebastian Hack. *Compiler Design: Syntactic and Semantic Analysis*. Springer, 2013.